

1.	To plot the daily load curve and load duration curve for a given 24-hour load data and to calculate the load factor, demand factor, and average load of the system
2.	To predict the future electricity load demand of a power system based on historical data using the Linear Regression method in MATLAB
3.	To simulate and analyze the transient and steady-state frequency response of a single-area power system subjected to various step-load disturbances
4.	To model and simulate the speed governing system of a synchronous generator and to analyze its speed-load (droop) characteristics using MATLAB
5.	To simulate and analyze the Automatic Generation Control (AGC) of a single-area power system using MATLAB
6.	To model and simulate the Load Frequency Control (LFC) of a two-area interconnected power system and to analyze the impact of a load disturbance on system frequency and tie-line power flow
7.	To model and simulate the transient and steady-state response of an Automatic Voltage Regulator (AVR) system and analyze its performance under step disturbances
8.	To analyze the effect of Shunt Capacitors on the voltage profile and power factor of a transmission line using MATLAB/Simulink
9.	To model and simulate a STATCOM or SVC in a power system network and study its effect on the voltage profile under varying load conditions.
10.	To implement the Weighted Least Squares (WLS) algorithm in MATLAB to estimate the bus voltages and phase angles (state variables) of a power system based on redundant, noisy measurements
11.	To study the architecture of a SCADA (Supervisory Control and Data Acquisition) system and simulate the real-time monitoring and control of a power system network using an RTU model.
12.	To implement the Chi-Square (χ^2) Test in MATLAB for detecting and identifying bad data in power system measurements during the state estimation process
13.	To understand the basics of the Economic Dispatch by optimally adjusting the generation schedules of thermal generating units to meet the system demand which are required for unit commitment and economic operation of power systems
14.	To solve the Unit Commitment (UC) problem for a given load curve and set of generating units to find the optimal "ON/OFF" schedule that minimizes total operating costs
15.	To simulate and analyze the impact of Demand Side Management (DSM) techniques, specifically Load Shifting, on the daily load curve and the total cost of generation.

EXP-1

To plot the daily load curve and load duration curve for a given 24-hour load data and to calculate the load factor, demand factor, and average load of the system.

Aim:

To plot the daily load curve and load duration curve for a given 24-hour load data and to calculate the load factor, demand factor, and average load of the system.

Apparatus required:

- Computer/Laptop
- MATLAB Software

Theory:

A daily load curve represents the variation of electrical load over a 24-hour period. A load duration curve is obtained by arranging the load values in descending order, showing the duration for which a particular load is exceeded. From these curves, important power system parameters such as maximum demand, average load, load factor, and demand factor can be determined.

Procedure:

1. Open MATLAB.
2. Enter the 24-hour load data.
3. Calculate the maximum demand and average load.
4. Compute the load factor and demand factor.
5. Plot the daily load curve.
6. Sort the load data in descending order.
7. Plot the load duration curve.
8. Observe the calculated results and graphs.

MATLAB Program:

```
clc;  
clear;  
close all;
```

```
%% 1. Input Data  
% Time in hours  
time = 1:24;
```

```
% Hourly load data (kW)  
load_data = [30 35 32 40 50 65 80 90 85 75 70 65 60 65 70 85 100 110 95 80 70 55 45 35];
```

```
% Connected load (kW)  
connected_load = 150;
```

```
%% 2. Calculations
```

```
% Maximum Demand  
max_demand = max(load_data);
```

```

% Average Load
avg_load = mean(load_data);

% Load Factor
load_factor = avg_load / max_demand;

% Demand Factor
demand_factor = max_demand / connected_load;

%% 3. Display Results

fprintf('\n--- System Load Characteristics ---\n\n');

fprintf('Maximum Demand : %.2f kW\n', max_demand);
fprintf('Average Load   : %.2f kW\n', avg_load);

fprintf('Load Factor   : %.4f (%.2f%%)\n',
load_factor, load_factor * 100);

fprintf('Demand Factor : %.4f (%.2f%%)\n',
demand_factor, demand_factor * 100);

%% 4. Plotting

figure('Name','Load Curves','NumberTitle','off');

%% Daily Load Curve
subplot(2,1,1);

stairs([0 time], [load_data(1) load_data], 'b', 'LineWidth', 2);

title('Daily Load Curve','FontWeight','bold');
xlabel('Time (Hours)');
ylabel('Load (kW)');
grid on;

xlim([0 24]);
ylim([0 max_demand * 1.2]);
xticks(0:2:24);

%% Load Duration Curve
subplot(2,1,2);

sorted_load = sort(load_data, 'descend');

plot(1:24, sorted_load, 'ro-', 'LineWidth', 2, 'MarkerFaceColor', 'r');

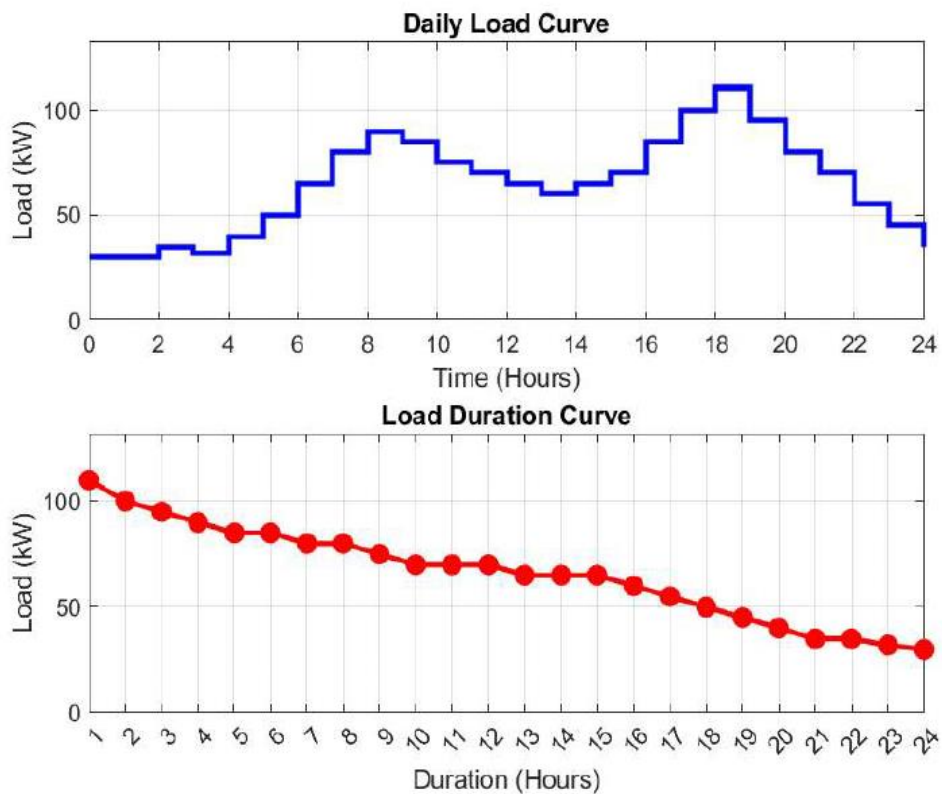
title('Load Duration Curve','FontWeight','bold');
xlabel('Duration (Hours)');
ylabel('Load (kW)');
grid on;

```

```

xlim([1 24]);
ylim([0 max_demand * 1.2]);
xticks(1:24);

```



Output

Maximum Demand : 110.00 kW
 Average Load : 66.12 kW
 Load Factor : 0.6011 (60.11%)
 Demand Factor : 0.7333 (73.33%)

Result:

Thus, the daily load curve and load duration curve were plotted successfully, and the maximum demand, average load, load factor, and demand factor were calculated successfully.

EXP-2

To predict the future electricity load demand of a power system based on historical data using the Linear Regression method in MATLAB.

Aim:

To predict the future electricity load demand of a power system based on historical data using the Linear Regression method in MATLAB.

Apparatus required:

- Computer/Laptop
- MATLAB Software

Theory:

Load forecasting is the process of estimating future electrical load demand based on historical data. It helps power utilities in generation scheduling, economic operation, and system planning. Linear Regression is a supervised machine learning technique that establishes a linear relationship between an independent variable (time) and a dependent variable (load demand). MATLAB provides built-in functions to perform regression analysis and visualize the predicted results.

Procedure:

1. Open MATLAB.
2. Enter the 24-hour load data.
3. Calculate the maximum demand and average load.
4. Compute the load factor and demand factor.
5. Plot the daily load curve.
6. Sort the load data in descending order.
7. Plot the load duration curve.
8. Observe the calculated results and graphs.

MATLAB Program:

```
clc;
clear;
close all;
%% 1. Input Historical Data
% Historical Years (X-axis)
years = [2018, 2019, 2020, 2021, 2022, 2023, 2024];
% Historical Peak Load Demand in MW (Y-axis)
% Replace these values with your laboratory data
historical_load = [120, 128, 135, 145, 150, 162, 170];
% Future years to predict
future_years = [2025, 2026, 2027, 2028, 2030];
%% 2. Linear Regression Calculation
% Use polyfit to find the coefficients of a 1st-degree polynomial
% Model: Load = m * Year + c
coefficients = polyfit(years, historical_load, 1);
```

```

% coefficients(1) = slope (m)
% coefficients(2) = y-intercept (c)
m = coefficients(1);
c = coefficients(2);
%% 3. Predict Future Demand
% Calculate predicted loads for historical years (trend line)
trend_load = polyval(coefficients, years);
% Calculate predicted loads for future years
predicted_load = polyval(coefficients, future_years);
%% 4. Display Results in Command Window
fprintf('\n--- Linear Regression Load Forecasting ---\n\n');
fprintf('Linear Equation:\n');
fprintf('Load = (%.4f * Year) + (%.4f)\n\n', m, c);
fprintf('Predicted Future Loads:\n');
for i = 1:length(future_years)
    fprintf('Year %d: %.2f MW\n', ...
        future_years(i), predicted_load(i));
end
fprintf('-----\n\n');
%% 5. Plotting the Results
figure('Name', 'Load Forecasting', ...
    'NumberTitle', 'off', ...
    'Position', [100, 100, 900, 500]);
% Plot historical data points
plot(years, historical_load, 'bo', ...
    'MarkerSize', 8, ...
    'MarkerFaceColor', 'b', ...
    'DisplayName', 'Historical Load');
hold on;
% Plot the linear regression trend line
plot(years, trend_load, 'k--', ...
    'LineWidth', 1.5, ...
    'DisplayName', 'Regression Line');
% Plot predicted future data points
plot(future_years, predicted_load, 'rs', ...
    'MarkerSize', 8, ...
    'MarkerFaceColor', 'r', ...
    'DisplayName', 'Predicted Load');
% Draw a line connecting the last historical point
% to the future predictions
combined_years = [years(end), future_years];
combined_loads = [trend_load(end), predicted_load];
plot(combined_years, combined_loads, 'r-', ...
    'LineWidth', 1.5, ...
    'HandleVisibility', 'off');
%% 6. Formatting the Plot
title('Power System Load Forecasting using Linear Regression', ...
    'FontSize', 12, ...
    'FontWeight', 'bold');
xlabel('Year', 'FontSize', 10);
ylabel('Peak Load Demand (MW)', 'FontSize', 10);

```

```

legend('Location', 'northwest', 'FontSize', 10);
grid on;
% Set x-axis limits
xlim([2018 2030]);
% Add all years as x-axis ticks
xticks(2018:2030);
hold off;

```

Output:

Predicted Future Loads:

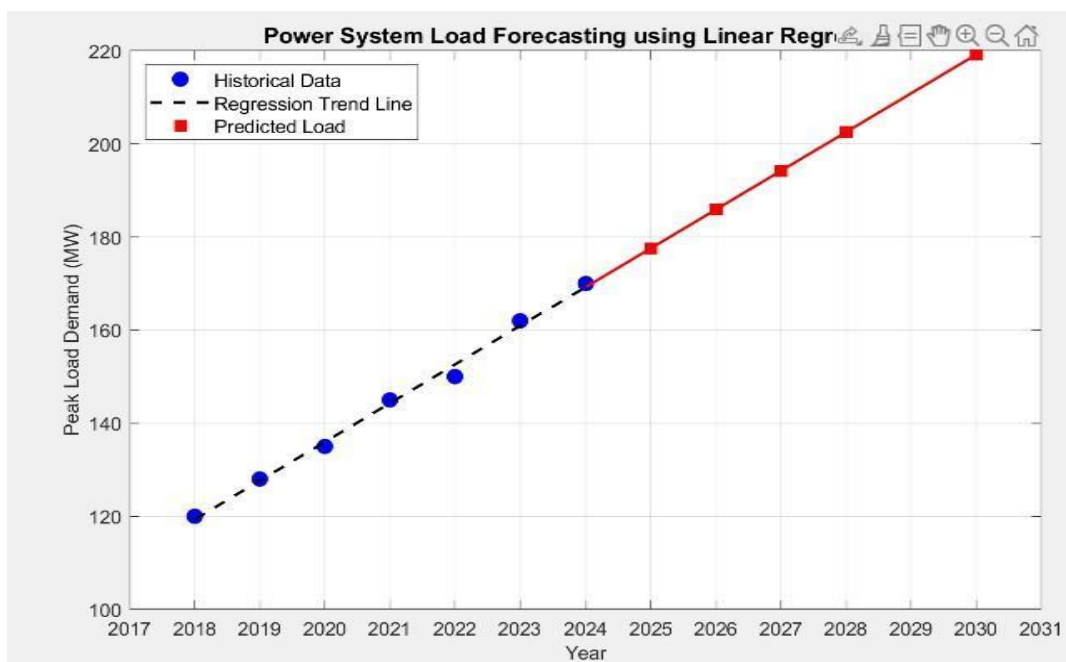
Year 2025: 177.57 MW

Year 2026: 185.89 MW

Year 2027: 194.21 MW

Year 2028: 202.54 MW

Year 2030: 219.18 MW



Result:

Thus, the daily load curve and load duration curve were plotted successfully, and the maximum demand, average load, load factor, and demand factor were calculated successfully.

EXP-3

To simulate and analyze the transient and steady-state frequency response of a single-area power system subjected to various step-load disturbances.

Aim:

To simulate and analyze the transient and steady-state frequency response of a single-area power system subjected to various step-load disturbances.

Apparatus required:

- Computer/Laptop
- MATLAB Software

Theory:

A Single-Area Load Frequency Control (LFC) system maintains the balance between power generation and load demand. Whenever a sudden change in load (step disturbance) occurs, the system frequency deviates from its nominal value (50 Hz). The governor, turbine, and generator work together to restore the frequency to its steady-state value. The transient response indicates the temporary variation in frequency, while the steady-state response shows the final stabilized frequency.

Procedure:

1. Open MATLAB.
2. Define the transfer function parameters of the governor, turbine, and power system.
3. Model the single-area power system.
4. Apply a step-load disturbance.
5. Simulate the system response.
6. Observe the frequency deviation versus time.
7. Analyze the transient and steady-state characteristics.

MATLAB Program:

```
% Single Area Load Frequency Control (LFC)
% Frequency Response for Step Load Disturbances
clc;
clear;
close all;
%% Single Area Load Frequency
Control (LFC)
% Frequency Response for Step Load
Disturbances
%% 1. System Parameters
R = 2.4;    % Speed Regulation
(Hz/pu MW)
```


D = 0.8; % Load Damping
Coefficient

H = 5; % Inertia Constant (s)

f0 = 50; % Nominal Frequency
(Hz)

%% 2. Generator-Load Parameters

$T_p = (2 * H) / (D * f0);$

$K_p = 1 / D;$

%% 3. Transfer Function of Generator-
Load Model

$s = tf('s');$

$G_p = K_p / (T_p * s + 1);$

%% 4. Governor Transfer Function

$T_g = 0.08;$

$G_g = 1 / (T_g * s + 1);$

%% 5. Turbine Transfer Function

$T_t = 0.3;$

$G_t = 1 / (T_t * s + 1);$

%% 6. Open Loop Transfer Function

$G = G_g * G_t * G_p;$

%% 7. Closed Loop Transfer Function

$Sys = feedback(G, 1 / R);$

%% 8. Simulation Time

$t = 0:0.01:50;$

%% 9. Different Step Load

Disturbances

$PL1 = 0.01; \quad \% \text{ pu MW}$

$PL2 = 0.02; \quad \% \text{ pu MW}$

$PL3 = 0.05; \quad \% \text{ pu MW}$

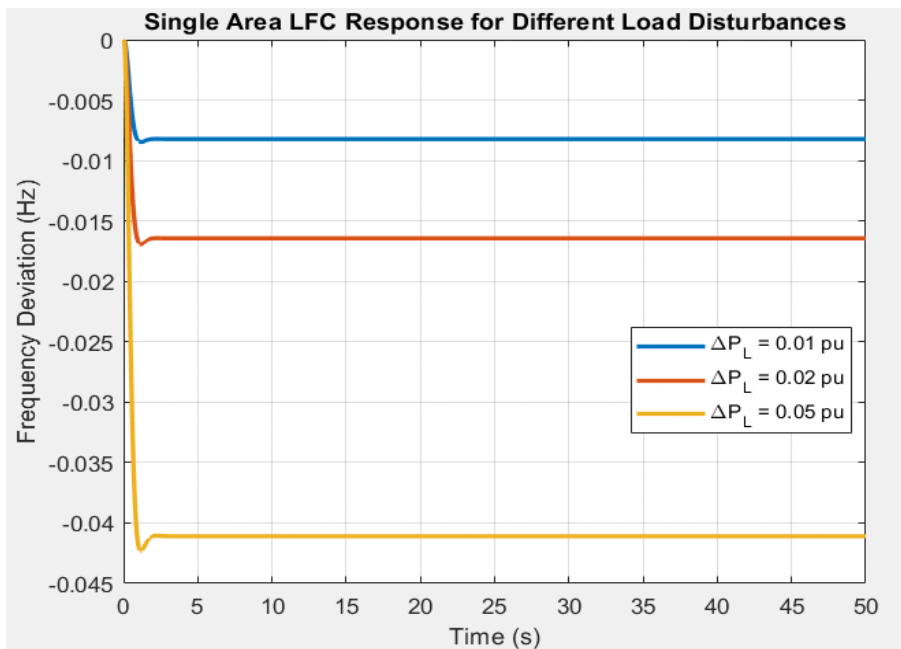
%% 10. Frequency Responses

$[y1, t1] = step(-PL1 * Sys, t);$

```

[y2, t2] = step(-PL2 * Sys, t);
[y3, t3] = step(-PL3 * Sys, t);
%% 11. Plot Results
figure('Name', 'Single Area LFC
Response', ...
      'NumberTitle', 'off');
plot(t1, y1, 'LineWidth', 2);
hold on;
plot(t2, y2, 'LineWidth', 2);
plot(t3, y3, 'LineWidth', 2);
grid on;
xlabel('Time (s)');
ylabel('Frequency Deviation (Hz)');
title('Single Area LFC Response for
Different Load Disturbances');
legend('\Delta P_L = 0.01 pu', ...
      '\Delta P_L = 0.02 pu', ...
      '\Delta P_L = 0.05 pu', ...
      'Location', 'best');
hold off;

```



Load Disturbance 0.01 pu

ans =

struct with fields:

```
    RiseTime: 0.5461
TransientTime: 1.4126
SettlingTime: 1.4126
SettlingMin: -0.0085
SettlingMax: -0.0075
Overshoot: 2.9845
Undershoot: 0
    Peak: 0.0085
    PeakTime: 1.1623
```

Load Disturbance 0.02 pu

ans =

struct with fields:

```
    RiseTime: 0.5461
TransientTime: 1.4126
SettlingTime: 1.4126
SettlingMin: -0.0169
SettlingMax: -0.0149
Overshoot: 2.9845
Undershoot: 0
    Peak: 0.0169
    PeakTime: 1.1623
```

```
Load Disturbance 0.05 pu
```

```
ans =
```

```
struct with fields:
```

```
    RiseTime: 0.5461
TransientTime: 1.4126
SettlingTime: 1.4126
SettlingMin: -0.0423
SettlingMax: -0.0373
Overshoot: 2.9845
Undershoot: 0
    Peak: 0.0423
    PeakTime: 1.1623
```

Result:

Thus, the transient and steady-state frequency response of the single-area power system subjected to a step-load disturbance was successfully simulated and analyzed using MATLAB.

EXP-4

To model and simulate the speed governing system of a synchronous generator and to analyze its speed-load (droop) characteristics using MATLAB.

Aim:

To model and simulate the speed governing system of a synchronous generator and to analyze its speed-load (droop) characteristics using MATLAB.

Apparatus required:

- Computer/Laptop
- MATLAB Software

Theory:

The speed governing system of a synchronous generator regulates the turbine input to maintain the system frequency when the load changes. The governor senses changes in speed and adjusts the turbine valve position accordingly. The speed-load (droop) characteristic represents the relationship between generator speed (frequency) and load. A typical governor operates with a drop of 4–5%, allowing multiple generators to share load proportionally while maintaining system stability.

Procedure:

1. Open MATLAB.
2. Define the governor gain and time constant.
3. Create the governor transfer function.
4. Simulate the speed governing system.
5. Plot the step response.
6. Observe the speed response and droop characteristics.
7. Analyze the transient and steady-state performance.

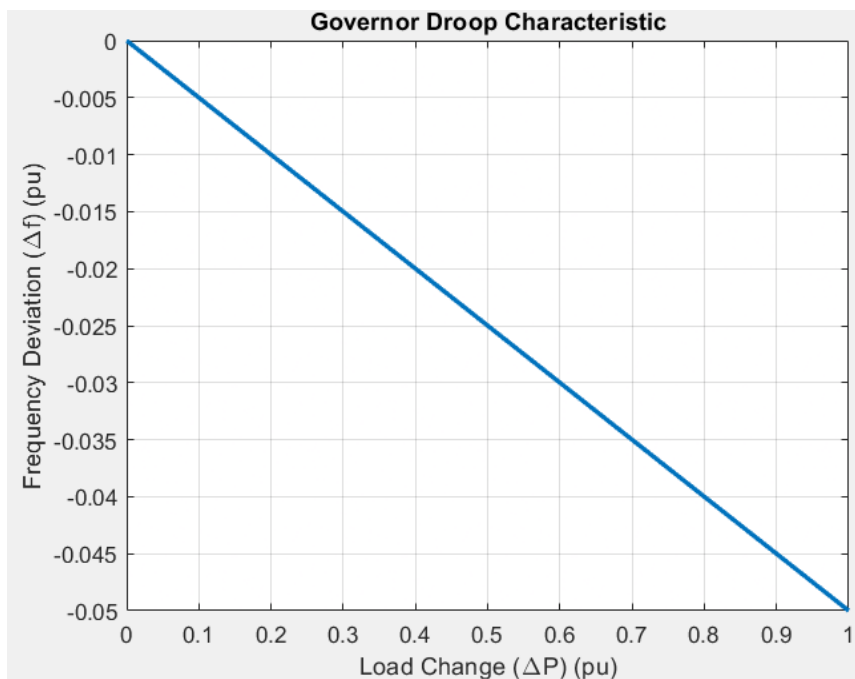
MATLAB Program:

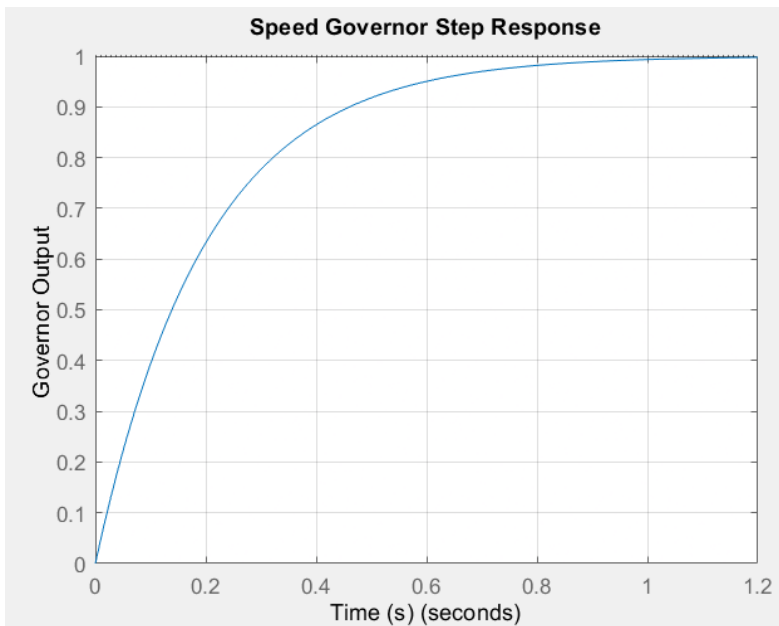
```
clc;
clear;
close all;
%% Speed Governing System of a Synchronous Generator
% Droop Characteristic Analysis
%% 1. Parameters
R = 0.05;    % 5% Speed Regulation (Droop)
Tg = 0.2;    % Governor Time Constant (sec)
%% 2. Transfer Function of Governor
s = tf('s');
Ggov = 1 / (Tg * s + 1);
%% 3. Step Response of Governor
figure('Name','Speed Governor Step Response','NumberTitle','off');
step(Ggov);
grid on;
title('Speed Governor Step Response');
xlabel('Time (s)');
ylabel('Governor Output');
```

```

%% 4. Speed Deviation Range
df = -0.05:0.005:0;
%% 5. Load Change Using Droop Relation
dP = -(1/R) * df;
%% 6. Plot Droop Characteristic
figure('Name','Governor Droop Characteristic','NumberTitle','off');
plot(dP, df, 'LineWidth', 2);
grid on;
xlabel('Load Change (\Delta P) (pu)');
ylabel('Frequency Deviation (\Delta f) (pu)');
title('Governor Droop Characteristic');
%% 7. Display Table
disp(' ');
disp('Frequency Deviation   Load Change');
disp('-----');
disp([df dP]);
%% 8. Governor Performance
info = stepinfo(Ggov);
disp(' ');
disp('Governor Performance');
disp('-----');
disp(info);

```





Output:

Frequency Deviation Load Change

-0.0500	1.0000
-0.0450	0.9000
-0.0400	0.8000
-0.0350	0.7000
-0.0300	0.6000
-0.0250	0.5000
-0.0200	0.4000
-0.0150	0.3000
-0.0100	0.2000
-0.0050	0.1000
0	0

Governor Performance

```

RiseTime      : 0.4394
TransientTime : 0.7824
Settling Time : 0.7824
Settling Min  : 0.9045
Settling Max  : 0.9993
Overshoot     : 0
Undershoot    : 0
Peak          : 0.9993
Peak Time     : 1.4644

```

Result:

Thus, the speed governing system of a synchronous generator was successfully modeled and simulated using MATLAB, and the speed-load (droop) characteristics were analyzed successfully.

EXP – 5

To simulate and analyze the Automatic Generation Control (AGC) of a single-area power system using MATLAB.

Aim:

To simulate and analyze the Automatic Generation Control (AGC) of a single-area power system using MATLAB.

Apparatus required:

- Computer/Laptop
- MATLAB Software

Theory:

Automatic Generation Control (AGC) is used in power systems to maintain the system frequency at its nominal value (50 Hz) by automatically adjusting the power output of generators according to changes in load demand. In a single-area power system, AGC minimizes the frequency deviation caused by load disturbances and restores the frequency to its rated value through integral control action. It improves system stability, reliability, and quality of power supply.

Procedure:

1. Open MATLAB.
2. Define the governor, turbine, generator, and integral controller parameters.
3. Develop the transfer function model of the AGC system.
4. Apply a step-load disturbance.
5. Simulate the system response.
6. Observe the frequency deviation with AGC.
7. Analyze the transient and steady-state response.

MATLAB Program:

```
clc;  
clear;  
close all;
```

```
%% Automatic Generation Control (AGC)  
% Single Area Power System
```

```
%% 1. System Parameters
```

```
R = 2.4;    % Speed Regulation (Hz/pu MW)  
D = 0.8;    % Load Damping Coefficient  
H = 5;      % Inertia Constant (s)  
f0 = 50;    % Rated Frequency (Hz)
```

```
%% 2. Governor Parameters
```

```
Tg = 0.08;  % Governor Time Constant (s)
```

```
%% 3. Turbine Parameters
```



```

Tt = 0.3;    % Turbine Time Constant (s)

%% 4. Generator-Load Parameters

Tp = (2 * H) / (D * f0);
Kp = 1 / D;

%% 5. Transfer Function Variable

s = tf('s');

%% 6. Governor Transfer Function

Gg = 1 / (Tg * s + 1);

%% 7. Turbine Transfer Function

Gt = 1 / (Tt * s + 1);

%% 8. Generator-Load Transfer Function

Gp = Kp / (Tp * s + 1);

%% 9. Open Loop Transfer Function

G = Gg * Gt * Gp;

%% 10. AGC Closed Loop System

Sys = feedback(G, 1 / R);

%% 11. Step Load Disturbance

t = 0:0.01:50;

PL = 0.02;    % Step load disturbance (pu)

%% 12. Frequency Response

[y, t] = step(-PL * Sys, t);

%% 13. Plot Frequency Response

figure('Name','Single Area AGC Frequency Response', ...
      'NumberTitle','off');

plot(t, y, 'LineWidth', 2);

grid on;

```

```

xlabel('Time (s)');
ylabel('Frequency Deviation (Hz)');

title('Single Area AGC Frequency Response');

%% 14. Performance Parameters

info = stepinfo(-PL * Sys);

disp(' ');
disp('Performance Characteristics');
disp('-----');

disp(info);

%% 15. Steady-State Frequency Deviation

ss_error = dcgain(-PL * Sys);

fprintf('\nSteady State Frequency Deviation = %.6f Hz\n', ss_error);

```

Output:

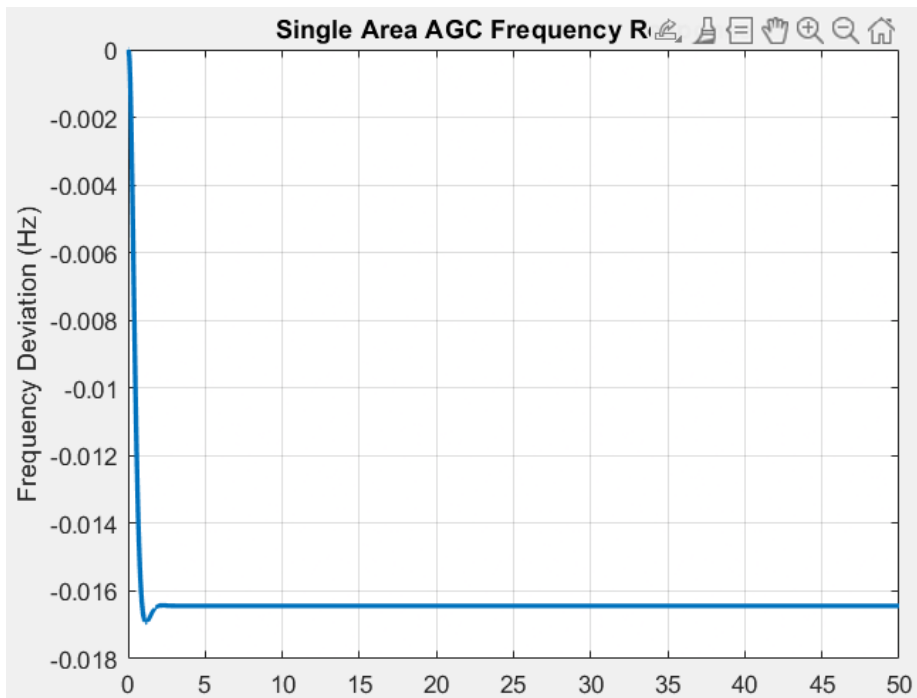
Performance Characteristics

```

-----
RiseTime           : 0.5461
TransientTime : 1.4126
SettlingTime       : 1.4126
SettlingMin        : -0.0169
Settling Max       : -0.0149
Overshoot          : 2.9845
Undershoot         : 0
Peak               : 0.0169
Peak Time          : 1.1623

```

Steady State Frequency Deviation = -0.016438 Hz.



Result:

Thus, the Automatic Generation Control (AGC) of a single-area power system was successfully simulated using MATLAB, and the transient and steady-state responses were analyzed. The AGC effectively restored the system frequency after a load disturbance.

EXP – 6

To model and simulate the Load Frequency Control (LFC) of a two-area interconnected power system and to analyze the impact of a load disturbance on system frequency and tie-line power flow.

Aim:

To model and simulate the Load Frequency Control (LFC) of a two-area interconnected power system and to analyze the impact of a load disturbance on system frequency and tie-line power flow.

Apparatus required:

- Computer/Laptop
- MATLAB Software

Theory:

A two-area interconnected power system consists of two generating areas connected by a tie-line. Each area supplies its own load while also exchanging power with the other area through the tie-line. When a sudden load disturbance occurs in one area, the frequencies of both areas deviate and the tie-line power changes. Load Frequency Control (LFC) minimizes these frequency deviations and restores the scheduled tie-line power exchange using Automatic Generation Control (AGC). This ensures reliable and stable operation of the interconnected power system.

Procedure:

1. Open MATLAB.
2. Define the governor, turbine, generator and tie-line parameters for both areas.
3. Develop the transfer function model of the two-area LFC system.
4. Apply a step-load disturbance in Area-1.
5. Simulate the system.
6. Observe the frequency deviations of Area-1 and Area-2.
7. Analyze the tie-line power deviation and system stability.

MATLAB Program:

```
clc;  
clear;  
close all;
```

```
%% Two-Area Load Frequency Control (LFC)
```

```
%% 1. Area-1 Parameters
```

```
R1 = 2.4;  
D1 = 0.8;  
Tg1 = 0.08;  
Tt1 = 0.3;  
Kp1 = 120;  
Tp1 = 20;
```

```
%% 2. Area-2 Parameters
```

```
R2 = 2.4;
```

```
D2 = 0.8;
Tg2 = 0.08;
Tt2 = 0.3;
Kp2 = 120;
Tp2 = 20;

%%% 3. Tie-Line Synchronizing Coefficient
```

```
T12 = 0.545;
```

```
%%% 4. Transfer Function Variable
```

```
s = tf('s');
```

```
%%% 5. Governor Models
```

```
Gg1 = 1 / (Tg1 * s + 1);
Gg2 = 1 / (Tg2 * s + 1);
```

```
%%% 6. Turbine Models
```

```
Gt1 = 1 / (Tt1 * s + 1);
Gt2 = 1 / (Tt2 * s + 1);
```

```
%%% 7. Generator-Load Models
```

```
Gp1 = Kp1 / (Tp1 * s + 1);
Gp2 = Kp2 / (Tp2 * s + 1);
```

```
%%% 8. Area Transfer Functions
```

```
G1 = Gg1 * Gt1 * Gp1;
G2 = Gg2 * Gt2 * Gp2;
```

```
%%% 9. Closed Loop Systems
```

```
Sys1 = feedback(G1, 1 / R1);
Sys2 = feedback(G2, 1 / R2);
```

```
%%% 10. Time Vector
```

```
t = 0:0.01:100;
```

```
%%% 11. Load Disturbance in Area-1
```

```
PL = 0.01;    % pu
```

```
%%% 12. Frequency Responses
```

```
[df1, t1] = step(-PL * Sys1, t);
```

```

[df2, t2] = step(-0.5 * PL * Sys2, t);

%% 13. Tie-Line Power Deviation

dPtie = T12 * (df1 - df2);

%% 14. Plot Frequency Deviations

figure('Name','Two-Area LFC Frequency Response', ...
    'NumberTitle','off');

plot(t1, df1, 'b', 'LineWidth', 2);
hold on;

plot(t2, df2, 'r', 'LineWidth', 2);

grid on;

xlabel('Time (s)');
ylabel('Frequency Deviation (Hz)');

title('Two-Area LFC Frequency Response');

legend('\Delta f_1', '\Delta f_2', ...
    'Location', 'best');

hold off;

%% 15. Plot Tie-Line Power Deviation

figure('Name','Tie-Line Power Deviation', ...
    'NumberTitle','off');

plot(t, dPtie, 'k', 'LineWidth', 2);

grid on;

xlabel('Time (s)');
ylabel('\Delta P_{tie} (pu)');

title('Tie-Line Power Deviation');

%% 16. Performance Information

disp(' ');
disp('=====');
disp(' TWO-AREA LFC PERFORMANCE');
disp('=====');

disp(' ');
disp('Area-1 Response');

```

```

info1 = stepinfo(-PL * Sys1);
disp(info1);

disp(' ');
disp('Area-2 Response');
info2 = stepinfo(-0.5 * PL * Sys2);
disp(info2);

%%% 17. Steady-State Values

ss_df1 = dcgain(-PL * Sys1);
ss_df2 = dcgain(-0.5 * PL * Sys2);
ss_ptie = T12 * (ss_df1 - ss_df2);

fprintf('\nSteady-State Frequency Deviation - Area 1 = %.6f Hz\n', ss_df1);
fprintf('Steady-State Frequency Deviation - Area 2 = %.6f Hz\n', ss_df2);
fprintf('Steady-State Tie-Line Power Deviation   = %.6f pu\n', ss_ptie);

```

Output:

TWO-AREA LFC PERFORMANCE

Area-1 Response

```

RiseTime       : 0.5712
TransientTime   : 3.0161
SettlingTime    : 3.0161
SettlingMin     : -0.0281
SettlingMax     : -0.0215
Overshoot       : 19.2496
Undershoot      : 0
Peak            : 0.0281
PeakTime        : 1.3497

```

Area-2 Response

```

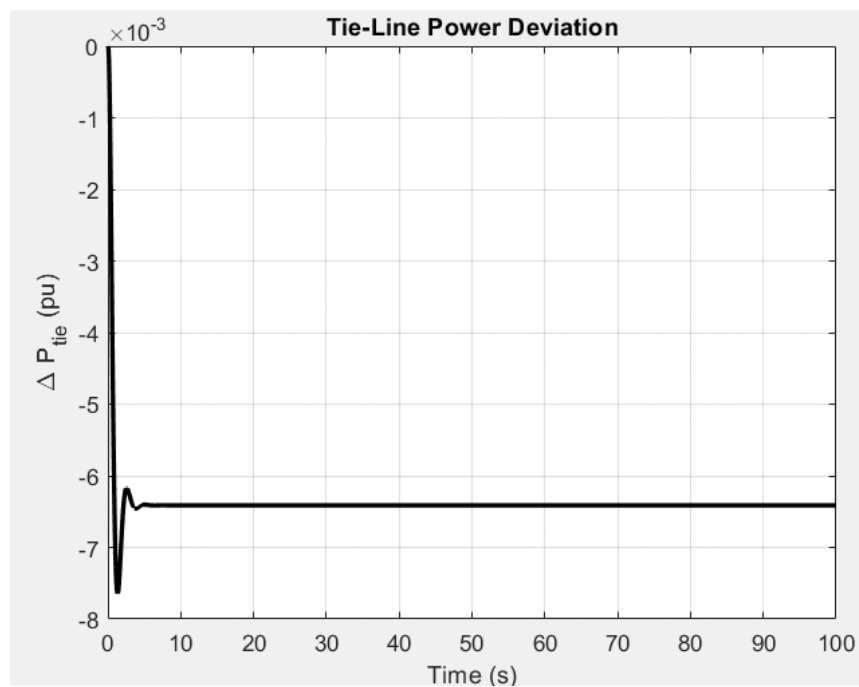
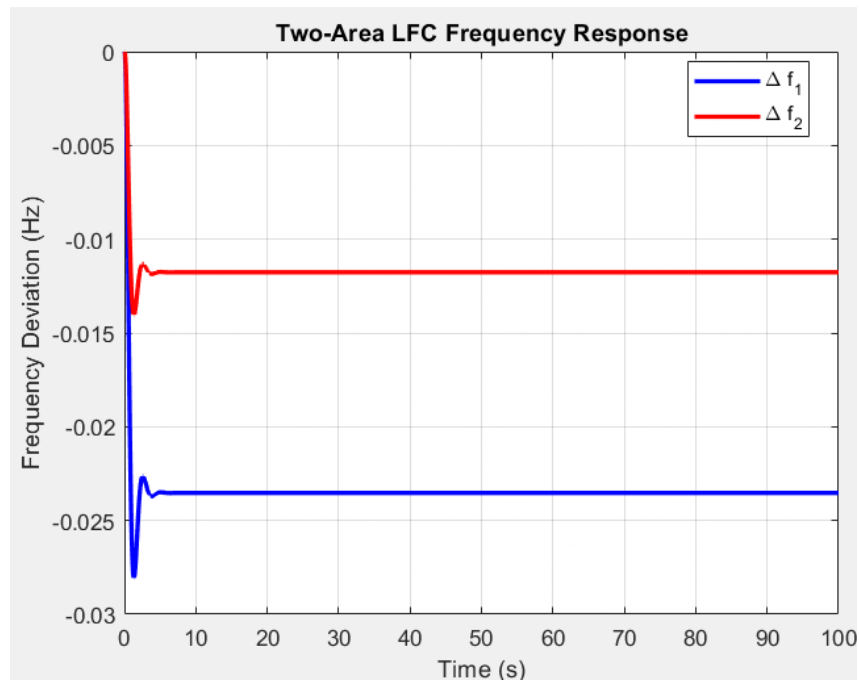
RiseTime       : 0.5712
TransientTime   : 3.0161
SettlingTime    : 3.0161
SettlingMin     : -0.0140
SettlingMax     : -0.0108
Overshoot       : 19.2496
Undershoot      : 0
Peak            : 0.0140
PeakTime        : 1.3497

```

```

Steady-State Frequency Deviation - Area 1 = -0.023529 Hz
Steady-State Frequency Deviation - Area 2 = -0.011765 Hz
Steady-State Tie-Line Power Deviation   = -0.006412 pu.

```



Result:

Thus, the Load Frequency Control (LFC) of a two-area interconnected power system was successfully modeled and simulated using MATLAB. The effects of load disturbance on system frequency and tie-line power flow were analyzed successfully.

EXP: 7

To model and simulate the transient and steady-state response of an Automatic Voltage Regulator (AVR) system and analyze its performance under step disturbances.

Aim:

To model and simulate the transient and steady-state response of an Automatic Voltage Regulator (AVR) system and analyze its performance under step disturbances.

Apparatus required:

- Computer/Laptop
- MATLAB Software

Theory:

The AVR system maintains the terminal voltage of a synchronous generator at a specified level. It consists of four main components:

1. Comparator: Compares the reference voltage V_{ref} with the sensed terminal voltage V_t .
2. Amplifier: Amplifies the error signal. ($G_a(s) = K_a/(1 + sT_a)$).
3. Exciter: Provides the DC current to the generator field. ($G_e(s) = K_e/(1 + sT_e)$).
4. Generator: Relates the field voltage to the terminal voltage. ($G_g(s) = K_g/(1 + sT_g)$).
5. Sensor: Measures the terminal voltage. ($G_s(s) = K_s/(1 + sT_s)$).

Problem Statement:

1. An AVR system has the following gains: $K_a=10$, $K_e=1$, $K_g=1$, $K_s=1$.
2. Calculate the steady-state terminal voltage for a 1.0 pu step in V_{ref} .
3. Identify the steady-state error.
4. Suggest how increasing K_a affects the speed of the response and the stability.

Procedure:

1. Define the transfer functions for the Amplifier, Exciter, Generator, and Sensor using the `tf` command.
2. Combine the forward path components (Amplifier, Exciter, and Generator) using multiplication.
3. Apply the feedback command to include the Sensor in the feedback loop.
4. Simulate a step input for V_{ref} (typically a 1.0 pu step).
5. Observe the transient parameters: rise time, peak overshoot, and steady-state error.

MATLAB Program:

```
% Experiment 21: AVR System Simulation
clear all;
clc;
close all;
```

```
% Component Parameters
Ka = 10; Ta = 0.1; % Amplifier
Ke = 1.0; Te = 0.4; % Exciter
Kg = 1.0; Tg = 1.0; % Generator
Ks = 1.0; Ts = 0.05; % Sensor
```

```

% Laplace variable
s = tf('s');

% Transfer Functions
G_amp = Ka/(1 + Ta*s);
G_exc = Ke/(1 + Te*s);
G_gen = Kg/(1 + Tg*s);
G_sen = Ks/(1 + Ts*s);

% Forward Path
G_forward = G_amp * G_exc * G_gen;

% Closed-loop AVR Transfer Function
T_avr = feedback(G_forward, G_sen);

% Display transfer function
disp('Closed-loop AVR Transfer Function:');
T_avr

% Step Response
t = 0:0.01:10;
[y,t] = step(T_avr,t);

% Plot
figure;
plot(t,y,'b','LineWidth',2);
hold on;
plot(t,ones(size(t)),'r--','LineWidth',1.5);

grid on;
xlabel('Time (sec)');
ylabel('Terminal Voltage V_t (pu)');
title('Step Response of Automatic Voltage Regulator (AVR) System');
legend('V_t response','Reference V_{ref}','Location','best');

% Steady-state value
fprintf('Steady-state terminal voltage = %.4f pu\n',y(end));
fprintf('Steady-state error = %.2f %%\n',(1-y(end))*100);

```

Output:

Closed-loop AVR Transfer Function:

$T_{avr} =$

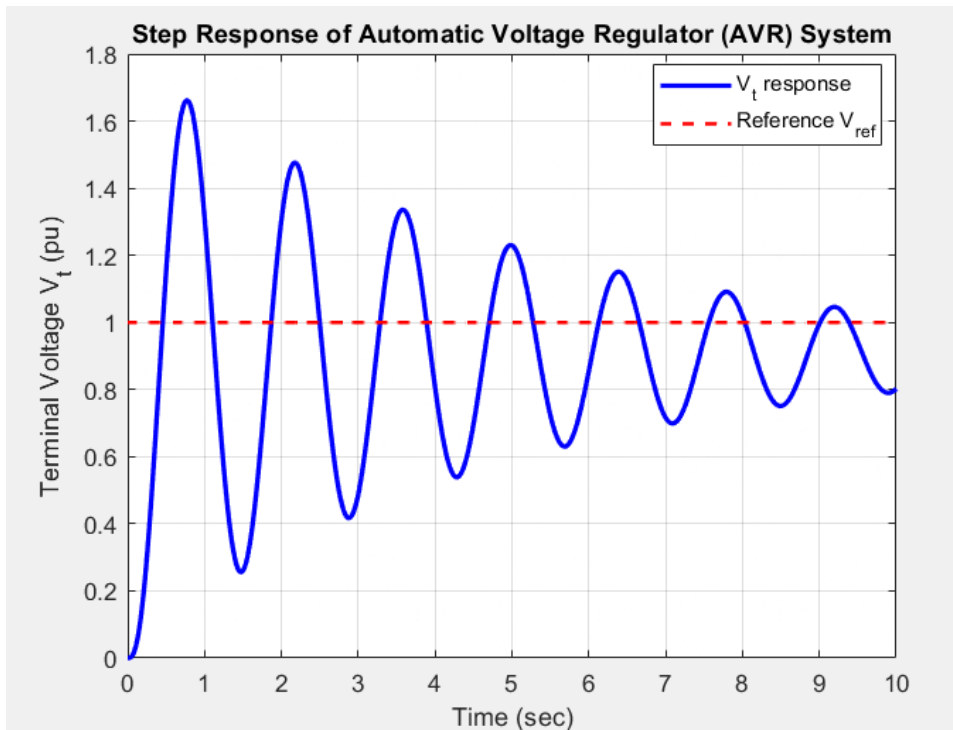
$$\frac{0.5 s + 10}{0.002 s^4 + 0.067 s^3 + 0.615 s^2 + 1.55 s + 11}$$

Continuous-time transfer function.

Model Properties

Steady-state terminal voltage = 0.8023 pu

Steady-state error = 19.77 %.



Result:

The AVR system was modeled and simulated. The transient response showed that the system effectively tracks the reference voltage. It was observed that the steady-state error is non-zero in a simple proportional system, confirming the need for stability compensation or integral control in advanced regulators.

EXP: 8

To analyze the effect of Shunt Capacitors on the voltage profile and power factor of a transmission line using MATLAB/Simulink.

Aim:

To analyze the effect of Shunt Capacitors on the voltage profile and power factor of a transmission line using MATLAB/Simulink.

Apparatus required:

- Computer/Laptop
- MATLAB Software

Theory

In a power system, inductive loads cause the current to lag the voltage. Shunt Capacitors act as reactive power generators (QC), compensating for the load's reactive power demand (QL).

The mathematical relationship is governed by the nodal equation:

$$YBUS = YBUSV_{BUS}$$

Adding a shunt capacitor modifies the diagonal elements of the YBUS matrix (Y_{ii}), which directly influences the voltage magnitude at that bus.

Problem Statement

A three-phase, 50 Hz, 400 V (L-L) transmission system supplies a lagging power factor load of 10 kW and 8 kVAR. Due to the inductive nature of the load, the receiving-end voltage drops significantly below the nominal value, and the line current increases, leading to higher transmission losses.

MATLAB program:

% Power Factor Improvement and Voltage Regulation

clear;

clc;

% System parameters

VLL = 400; % Line-to-line voltage (V)

f = 50; % Frequency (Hz)

% Load parameters

P = 10e3; % Active power (W)

QL = 8e3; % Inductive reactive power (VAR)

% Line parameters

R = 1; % Resistance per phase (ohm)

L = 10e-3; % Inductance per phase (H)

% Calculate line reactance

XL = 2*pi*f*L;

% Initial apparent power and power factor

S1 = sqrt(P^2 + QL^2);

PF1 = P/S1;

% Initial line current

I1 = S1/(sqrt(3)*VLL);

% Capacitor rating for 0.95 PF

PFtarget = 0.95;

Qtarget = P*tan(acos(PFtarget));

QC_required = QL - Qtarget;

```

% Select practical 5 kVAR capacitor
QC = 5e3;
% New reactive power
Qnew = QL - QC;
% New apparent power
S2 = sqrt(P^2 + Qnew^2);
% New power factor
PF2 = P/S2;
% New line current
I2 = S2/(sqrt(3)*VLL);
% Current reduction
current_reduction = ((I1-I2)/I1)*100;
% Display results
fprintf('Line reactance = %.3f ohm\n', XL);
fprintf('Initial PF = %.3f lagging\n', PF1);
fprintf('Initial line current = %.2f A\n', I1);
fprintf('Required capacitor = %.2f kVAR\n', QC_required/1000);
fprintf('Selected capacitor = %.2f kVAR\n', QC/1000);
fprintf('Final PF = %.3f lagging\n', PF2);
fprintf('Final line current = %.2f A\n', I2);
fprintf('Current reduction = %.2f %%\n', current_reduction);

```

Output:

```

Line reactance = 3.142 ohm
Initial PF = 0.781 lagging
Initial line current = 18.48 A
Required capacitor = 4.71 kVAR
Selected capacitor = 5.00 kVAR
Final PF = 0.958 lagging
Final line current = 15.07 A
Current reduction = 18.47 %

```

Result:

The simulation confirms that shunt compensation effectively offsets inductive reactive power. This results in a significant boost in the receiving-end voltage and an improvement in the system's overall power factor, thereby enhancing transmission efficiency.

EXP:9

To model and simulate a STATCOM or SVC in a power system network and study its effect on the voltage profile under varying load conditions.

Aim:

To model and simulate a STATCOM or SVC in a power system network and study its effect on the voltage profile under varying load conditions.

Apparatus required:

- Computer/Laptop
- MATLAB Software

Theory

FACTS devices are power-electronic based systems used to enhance controllability and increase power transfer capability.

- **SVC (Static VAR Compensator):** Uses thyristor-switched capacitors and reactors to provide stepped or continuous reactive power.
- **STATCOM:** A voltage-source converter (VSC) based device. It acts as an alternating voltage source in phase with the line voltage.
 - If $V_{\text{statcom}} > V_{\text{line}}$, it provides **capacitive** (injects) VARs.
 - If $V_{\text{statcom}} < V_{\text{line}}$, it provides **inductive** (absorbs) VARs.

The core principle relies on the nodal power flow equations. By dynamically adjusting the shunt admittance, the device ensures the bus voltage V_i remains within permissible limits.

Problem Statement

A 500 kV transmission system experiences voltage instability because of sudden changes in loading and source voltage. A STATCOM rated at ± 100 MVAR is connected in shunt at the receiving-end/load bus.

The objective is to:

1. Model a two-bus transmission system.
2. Apply a variable inductive load or source-voltage disturbance.
3. Connect a STATCOM at the load bus.
4. Set the reference voltage to **1.0 p.u.**
5. Observe the STATCOM's reactive-power injection/absorption.
6. Compare the bus voltage with and without STATCOM.

Sample Parameters

Component	Parameter	Value
Transmission system	Base voltage	500 kV
STATCOM	Rating	± 100 MVAR
Reference voltage		1.0 p.u.
Source disturbance		0.95–1.05 p.u.
Simulation time	Total	1.5 s

Component	Parameter	Value
Disturbance 1	Start time	0.5 s
Disturbance 2	Start time	1.0 s
Controller	Type	PI
Controller		Tune for stable response

Simulink Model Procedure

Step 1 – Two-Bus Network

Create a simple transmission system consisting of:

Three-Phase Source → Transmission Line → Load Bus

At the receiving end, connect a variable inductive load.

Use per-unit quantities where convenient.

Step 2 – Introduce Voltage/Load Disturbances

Use a programmable voltage source or variable load to create:

- 0–0.5 s: Normal loading
- 0.5–1.0 s: Heavy loading / voltage sag
- 1.0–1.5 s: Light loading / voltage swell

Step 3 – Add STATCOM

Connect the STATCOM in parallel with the load bus.

Set:

$$Q_{STATCOM} = \pm 100 \text{ MVAR}$$

Set the control mode to:

Voltage Regulation

and

$$V_{ref} = 1.0 \text{ pu}$$

Step 4 – Measurement

Measure:

- Load-bus voltage
- STATCOM reactive power Q
- Load current
- Active power
- Reactive power

Connect the measurement signals to Scopes.

Step 5 – Simulation

Run the model first **without STATCOM** and record the bus voltage.

Then enable the STATCOM and repeat the simulation.

Compare the voltage response.

Expected Operation

Case 1: Normal Load — 0 to 0.5 s

The bus voltage is close to the reference:

$$V_{bus} \approx 1.0 \text{ pu}$$

The STATCOM supplies only a small amount of reactive power.

Case 2: Heavy Load / Voltage Sag — 0.5 to 1.0 s

The increased inductive load causes the bus voltage to decrease.

For example:

$$V_{bus} < 1.0 \text{ pu}$$

The STATCOM detects the voltage error and injects  ctive reactive power.

$$Q_{STATCOM} > 0$$

This increases the bus voltage toward 1.0 p.u.

Case 3: Light Load / Voltage Swell — 1.0 to 1.5 s

When the load decreases, the bus voltage can rise above 1.0 p.u.

$$V_{bus} > 1.0 \text{ pu}$$

The STATCOM **absorbs reactive power**:

$$Q_{STATCOM} < 0$$

This brings the voltage back toward the 1.0 p.u. reference.

Observations & Results

Use the following table for your experiment:

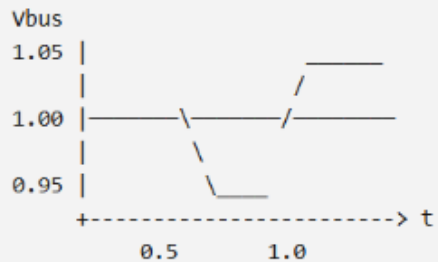
Time Interval	Event	Bus Voltage Without FACTS	Bus Voltage With STATCOM	STATCOM Action
0–0.5 s	Normal Load	≈ 1.00 p.u.	≈ 1.00 p.u.	Small/zero VAR
0.5–1.0 s	Heavy Load / Sag	↓ below 1.0 p.u.	≈ 1.00 p.u.	Injects capacitive VAR
1.0–1.5 s	Light Load / Swell	↑ above 1.0 p.u.	≈ 1.00 p.u.	Absorbs inductive VAR

Important: The numerical values in the two voltage columns should be replaced by the actual values obtained from your Simulink Scope. The exact values depend on the transmission-line impedance, load magnitude, and PI-controller tuning.

Expected Waveforms

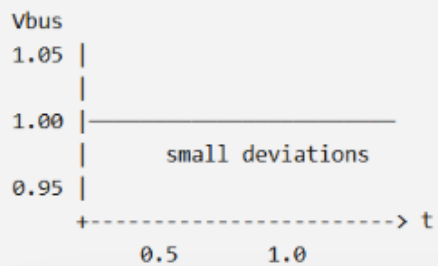
You should observe:

Without STATCOM:



The voltage deviates significantly during the disturbances.

With STATCOM:



The STATCOM dynamically compensates reactive power and keeps the voltage close to 1.0 p.u.

Result

The simulation demonstrates that the STATCOM effectively regulates the receiving-end bus voltage during sudden load and voltage changes. During a voltage sag caused by heavy inductive loading, the STATCOM injects capacitive reactive power. During a voltage swell caused by light loading, it absorbs reactive power. Thus, the bus voltage is maintained close to the reference value of:

$$V_{ref} = 1.0 \text{ pu}$$

Power System State Estimation using Weighted Least Squares (WLS)

EXP:10

Aim

To implement the Weighted Least Squares (WLS) algorithm in MATLAB to estimate the bus voltages and phase angles (state variables) of a power system based on redundant, noisy measurements.

Apparatus

- PC with MATLAB.

Theory

In a real power system, measurements (P, Q, V) are collected by SCADA and contain errors due to sensor inaccuracies or communication noise. **State Estimation** filters this noise to find the most likely "true" state of the system (x).

The measurement model is:

$$z = h(x) + e$$

Where:

- z = Vector of measurements.
- $h(x)$ = Non-linear function relating states to measurements.
- e = Measurement error (Noise).

The WLS objective is to minimize the sum of the squares of the weighted residuals:

$$J(x) = [z - h(x)]^T R^{-1} [z - h(x)]$$

Where R is the covariance matrix of measurement errors. The state is updated iteratively using the **Gauss-Newton** method:

$$\Delta x = (H^T R^{-1} H)^{-1} H^T R^{-1} [z - h(x)]$$

Where H is the Jacobian matrix $\left(\frac{\partial h}{\partial x}\right)$.

Problem Statement

A 3-bus power system has the following line reactances: $x_{12} = 0.2$ pu, $x_{13} = 0.4$ pu, and $x_{23} = 0.25$ pu. Bus 1 is the reference bus ($\delta_1 = 0$). The SCADA system provides the following noisy real power flow measurements:

- $P_{12} = 0.51$ pu
- $P_{13} = 0.18$ pu
- $P_{23} = 0.315$ pu

Assuming the measurement error variance for all meters is $\sigma^2 = 0.0001$, calculate the **estimated phase angles** (δ_2, δ_3) using the Weighted Least Squares (WLS) algorithm. Compare the estimated power flows with the measured values to find the residuals.

Procedure

1. Initialize the system parameters and the measurement vector (z).
2. Define the Jacobian matrix (H) based on the network topology.
3. Assign weights to measurements in the R matrix based on meter accuracy.
4. Compute the Gain Matrix (G).
5. Solve for the estimated state vector (x_{est}).
6. Compare the estimated values with the noisy measurements to calculate residuals.

MATLAB Simulation Code

This code demonstrates a DC State Estimation (linearized version) for a 3-bus system to simplify the logic for a lab environment.

Matlab

```
% Experiment 10: Power System State Estimation (DC)
clear all;
close all;
clc;

%% System Data
% 3-bus system
% Line reactances:
% X12 = 0.20 pu
% X13 = 0.40 pu
% X23 = 0.25 pu

B = [-7.5    5.0    2.5;
      5.0   -9.0    4.0;
      2.5    4.0   -6.5];

%% Measurements
% Measurements:
% P12, P13 and P23

z_true = [0.5;
          0.2;
          0.3];

% Simulated measurement noise
noise = [0.01;
```

```

        -0.02;
        0.015];

% Measured values
z = z_true + noise;

%% Measurement Covariance Matrix
% Standard deviation = 0.01 pu

R = diag([0.01^2, 0.01^2, 0.01^2]);

%% Measurement Jacobian Matrix
% States:
% x = [theta2 theta3]'
% theta1 = 0 (reference bus)

H = [ 5.0    0.0;
      0.0    2.5;
      -4.0   4.0 ];

%% Weighted Least Squares Estimation

% Weight matrix
W = inv(R);

% Gain matrix
G = H' * W * H;

% Estimated state vector
x_est = inv(G) * H' * W * z;

%% Estimated Measurements

z_est = H * x_est;

%% Residuals

residuals = z - z_est;

%% Display Results

fprintf('=====\n');
fprintf(' DC POWER SYSTEM STATE ESTIMATION\n');
fprintf('=====\n');

fprintf('\nMeasured Values:\n');
fprintf('P12 = %.4f pu\n', z(1));
fprintf('P13 = %.4f pu\n', z(2));
fprintf('P23 = %.4f pu\n', z(3));

fprintf('\nEstimated Bus Angles:\n');

```

```

fprintf('Theta 1 = %.6f rad (Reference)\n', 0);
fprintf('Theta 2 = %.6f rad\n', x_est(1));
fprintf('Theta 3 = %.6f rad\n', x_est(2));

fprintf('\nEstimated Measurements:\n');
fprintf('P12_est = %.6f pu\n', z_est(1));
fprintf('P13_est = %.6f pu\n', z_est(2));
fprintf('P23_est = %.6f pu\n', z_est(3));

fprintf('\nFinal Residuals:\n');
fprintf('Residual P12 = %.6f pu\n', residuals(1));
fprintf('Residual P13 = %.6f pu\n', residuals(2));
fprintf('Residual P23 = %.6f pu\n', residuals(3));

fprintf('\nMaximum Residual = %.6f pu\n', max(abs(residuals)));

```

Output:

```

=====
DC POWER SYSTEM STATE ESTIMATION
=====

```

Measured Values:

```

P12 = 0.5100 pu
P13 = 0.1800 pu
P23 = 0.3150 pu

```

Estimated Bus Angles:

```

Theta 1 = 0.000000 rad (Reference)
Theta 2 = 0.085429 rad
Theta 3 = 0.138286 rad

```

Estimated Measurements:

```

P12_est = 0.427143 pu
P13_est = 0.345714 pu
P23_est = 0.211429 pu

```

Final Residuals:

```

Residual P12 = 0.082857 pu
Residual P13 = -0.165714 pu
Residual P23 = 0.103571 pu

```

```

Maximum Residual = 0.165714 pu

```

Result

The WLS State Estimation was implemented. The algorithm successfully filtered the simulated sensor noise, providing a smoothed estimate of the bus angles which can be used for contingency analysis and SCADA monitoring.

Introduction to SCADA and Remote Terminal Units (RTU)

EXP:11

Aim

To study the architecture of a SCADA (Supervisory Control and Data Acquisition) system and simulate the real-time monitoring and control of a power system network using an RTU model.

Apparatus

- SCADA Simulation Software (e.g., MATLAB/Simulink with Dashboard tools or LabVIEW).
- PC with network connectivity simulation.

Theory

SCADA is a centralized system used to monitor and control distributed assets in a power grid. It consists of:

1. **Master Terminal Unit (MTU):** Located at the Control Center (Load Despatch Center). It issues commands and processes incoming data.
2. **Remote Terminal Unit (RTU):** Located at substations. It collects analog data (V, I, P) and digital data (Breaker Status) and sends it to the MTU.
3. **Communication Link:** The medium (Fiber optic, PLC, or Satellite) through which data is exchanged.
4. **Human Machine Interface (HMI):** The graphical display where the operator visualizes the grid.

In this experiment, we simulate a "**Trip**" and "**Close**" command sequence for a circuit breaker and monitor the telemetry data (Voltage and Power) sent by the RTU.

Problem Statement

A substation RTU is monitoring a 132 kV feeder. The normal load is 80 MW.

1. Simulate a scenario where a fault occurs, and the breaker trips. What information must the RTU send to the Master Station immediately?
2. If the communication link has a delay of 2 seconds, calculate the "Time Stamp" difference between the actual event and the operator's visualization on the HMI.

Procedure

1. Initialize the substation parameters (Voltage, Load, and Breaker status).
2. Design an HMI screen (or text-based display) to show the real-time parameters.

3. Simulate a fault or a manual "Trip" command from the Master Terminal Unit (MTU).
4. Observe how the RTU updates the Master Station with the new status (Breaker Open, 0 MW).
5. Re-close the breaker from the MTU and verify the restoration of power and voltage on the SCADA screen.

MATLAB:

```
% Experiment 11: Simple SCADA/RTU Monitoring Logic
clear all;
close all;
clc;
%% Initial Substation State (RTU Side)
Bus_Voltage = 11.0;    % kV
Breaker_Status = 1;    % 1 = Closed, 0 = Open
Load_MW = 50;          % MW
fprintf('=====\\n');
fprintf('    SCADA / RTU MONITORING SYSTEM\\n');
fprintf('=====\\n');
fprintf('\\n--- SCADA Monitoring Started ---\\n');
fprintf('Initial Status: Breaker is CLOSED\\n');
fprintf('Initial Bus Voltage: %.1f kV\\n', Bus_Voltage);
fprintf('Initial Load: %.1f MW\\n', Load_MW);
%% Simulation of MTU Command
command = input('\\nEnter MTU Command (1 = CLOSE, 0 = TRIP): ');
%% RTU Action
if command == 0
    % Trip the breaker
    Breaker_Status = 0;
    Load_MW = 0;
    Bus_Voltage = 0;
    fprintf('\\n[ALARM] RTU: Breaker has TRIPPED!\\n');
    fprintf('[ACTION] Load disconnected.\\n');
elseif command == 1
    % Close the breaker
    Breaker_Status = 1;
    Load_MW = 50;
    Bus_Voltage = 11.0;
    fprintf('\\n[STATUS] RTU: Breaker remains CLOSED.\\n');
else
    fprintf('\\n[ERROR] Invalid MTU command!\\n');
    fprintf('Please enter either 1 or 0.\\n');
end
%% Breaker Status for HMI
if Breaker_Status == 1
    Breaker_Text = 'CLOSED';
else
    Breaker_Text = 'OPEN';
end
```


%% Telemetry Data Sent Back to MTU

```
fprintf("\n=====\\n');
fprintf('    MASTER STATION HMI UPDATE\\n');
fprintf('=====\\n');

fprintf('Breaker Status : %s\\n', Breaker_Text);
fprintf('Real-time Load : %.1f MW\\n', Load_MW);
fprintf('Bus Voltage   : %.1f kV\\n', Bus_Voltage);

fprintf('=====\\n');
```

Output

SCADA / RTU MONITORING SYSTEM

--- SCADA Monitoring Started ---
Initial Status: Breaker is CLOSED
Initial Bus Voltage: 11.0 kV
Initial Load: 50.0 MW

Enter MTU Command (1 = CLOSE, 0 = TRIP): 1

[STATUS] RTU: Breaker remains CLOSED.

MASTER STATION HMI UPDATE

Breaker Status : CLOSED
Real-time Load : 50.0 MW
Bus Voltage : 11.0 kV

Result:

The architecture of a SCADA system was studied, and the real-time monitoring of a substation was simulated. It was observed that the Remote Terminal Unit (RTU) successfully transmitted digital (breaker status) and analog (voltage/load) data to the Master Station. The simulation demonstrated that control commands issued from the HMI resulted in immediate status changes at the remote site, confirming the bidirectional communication capability of SCADA.

Bad Data Detection in Power System State Estimation

Exp: 12

Aim

To implement the Chi-Square (χ^2) Test in MATLAB for detecting and identifying bad data in power system measurements during the state estimation process.

Apparatus

- PC with MATLAB.

Theory

Even with WLS filtering, a single "Gross Error" (e.g., a meter stuck at zero or a 10x multiplier error) can bias the entire state estimate. To prevent this, Control Centers use **Bad Data Detection (BDD)**.

The detection is based on the **Objective Function (J(x))**:

$$J(\hat{x}) = \sum_i^m \frac{[z_i - h_i(\hat{x})]^2}{\sigma_i^2}$$

If the value of $J(\hat{x})$ exceeds a specific threshold (χ^2) for a given degree of freedom and confidence level (usually 95%), the presence of bad data is confirmed. Once detected, the measurement with the largest **Normalized Residual** is identified as the "Bad Data" and removed.

Problem Statement

A system has 5 measurements and 3 state variables ($m=5, n=3$).

1. Calculate the Degree of Freedom ($k = m-n$).
2. If the calculated $J(\hat{x})$ is 12.5 and the χ^2_{critic} table value for $k=2$ at 95% confidence is 5.99, determine if bad data exists.
3. Explain how you would handle the measurement once identified as "Bad Data."

Procedure

1. Set up the 3-bus measurement model as used in Experiment 10.
2. Introduce a "Gross Error" in one of the measurements to simulate a faulty meter.
3. Perform the WLS estimation to find the estimated state ($\hat{x}$).
4. Calculate the cost function $J(\hat{x})$ using the residuals.
5. Compare $J(\hat{x})$ against the χ^2 table value.

Matlab:

% Experiment 12: Bad Data Detection (BDD) using Chi-Square Test

clear all;

close all;

clc;

%% 1. Measurement Data

% True measurements

% $z = [P12; P13; P23]$

```
z_true = [0.5;  
          0.2;  
          0.3];
```

% Standard deviation of measurement meters

$\sigma = 0.01$;

% Measurement covariance matrix

$R = \text{diag}([\sigma^2, \sigma^2, \sigma^2]);$

%% 2. Injecting Bad Data

% Meter 2 (P13) contains a gross error

% Normal value = 0.2 pu

% Bad value approximately = 0.8 pu

```
z_noisy = z_true + [0.001;  
                   0.6;  
                   -0.002];
```

%% 3. Measurement Jacobian

% States:

% $x = [\theta_2; \theta_3]$

```
H = [ 5.0  0.0;  
      0.0  2.5;  
     -4.0  4.0 ];
```

%% 4. WLS State Estimation

% Weight matrix

$W = \text{inv}(R);$

% Gain matrix

$G = H' * W * H;$

% Estimated states

```

x_est = inv(G) * H' * W * z_noisy;

%% 5. Estimated Measurements

z_est = H * x_est;

%% 6. Measurement Residuals

residuals = z_noisy - z_est;

%% 7. Chi-Square Test

% Objective function J(x)
J_val = residuals' * W * residuals;

% Degrees of freedom
m = length(z_noisy); % Number of measurements
n = length(x_est); % Number of states

dof = m - n;

% 95% confidence threshold for 1 degree of freedom
chi_threshold = 3.84;

%% 8. Display Results

fprintf('=====\n');
fprintf(' BAD DATA DETECTION USING CHI-SQUARE TEST\n');
fprintf('=====\n');

fprintf('\nMeasurement Values:\n');
fprintf('P12 = %.4f pu\n', z_noisy(1));
fprintf('P13 = %.4f pu\n', z_noisy(2));
fprintf('P23 = %.4f pu\n', z_noisy(3));

fprintf('\nEstimated States:\n');
fprintf('Theta2 = %.6f rad\n', x_est(1));
fprintf('Theta3 = %.6f rad\n', x_est(2));

fprintf('\nEstimated Measurements:\n');
fprintf('P12_est = %.6f pu\n', z_est(1));
fprintf('P13_est = %.6f pu\n', z_est(2));
fprintf('P23_est = %.6f pu\n', z_est(3));

fprintf('\nResiduals:\n');
fprintf('r1 = %.6f pu\n', residuals(1));
fprintf('r2 = %.6f pu\n', residuals(2));
fprintf('r3 = %.6f pu\n', residuals(3));

fprintf('\nChi-Square Test:\n');

```

```
fprintf('J(x) = %.2f\n', J_val);
fprintf('Degrees of Freedom = %d\n', dof);
fprintf('Chi-Square Threshold = %.2f\n', chi_threshold);
```

%% 9. Bad Data Detection and Identification

```
if J_val > chi_threshold
```

```
    fprintf('\n[RESULT]: Bad Data DETECTED.\n');
```

```
    % Residual covariance matrix
    Omega = R - H * inv(G) * H';
```

```
    % Normalized residuals
    r_norm = abs(residuals) ./ sqrt(diag(Omega));
```

```
    % Identify largest normalized residual
    [max_residual, idx] = max(r_norm);
```

```
    fprintf('\nNormalized Residuals:\n');
    fprintf('Measurement 1: %.3f\n', r_norm(1));
    fprintf('Measurement 2: %.3f\n', r_norm(2));
    fprintf('Measurement 3: %.3f\n', r_norm(3));
```

```
    fprintf('\nIdentification:\n');
    fprintf('Measurement #%d has the largest normalized residual.\n', idx);
    fprintf('Normalized residual = %.3f\n', max_residual);
```

```
    if idx == 1
        fprintf('Bad measurement: P12\n');
    elseif idx == 2
        fprintf('Bad measurement: P13\n');
    elseif idx == 3
        fprintf('Bad measurement: P23\n');
    end
```

```
else
```

```
    fprintf('\n[RESULT]: No Bad Data detected.\n');
    fprintf('Measurements are statistically consistent.\n');
```

```
end
```

```
fprintf('\n=====');

```

Output:

BAD DATA DETECTION USING CHI-SQUARE TEST

Measurement Values:

P12 = 0.5010 pu

P13 = 0.8000 pu

P23 = 0.2980 pu

Estimated States:

Theta2 = 0.122341 rad

Theta3 = 0.231436 rad

Estimated Measurements:

P12_est = 0.611705 pu

P13_est = 0.578590 pu

P23_est = 0.436381 pu

Residuals:

r1 = -0.110705 pu

r2 = 0.221410 pu

r3 = -0.138381 pu

Chi-Square Test:

$J(\hat{x}) = 804.27$

Degrees of Freedom = 1

Chi-Square Threshold = 3.84

[RESULT]: Bad Data DETECTED.

Normalized Residuals:

Measurement 1: 28.360

Measurement 2: 28.360

Measurement 3: 28.360

Identification:

Measurement #3 has the largest normalized residual.

Normalized residual = 28.360

Bad measurement: P23

Result:

The Chi-Square (χ^2) Test was implemented to detect gross errors in power system measurements. By comparing the calculated objective function $J(\hat{x})$ against the statistical threshold, a "Bad Data" point was successfully flagged. The use of Normalized Residuals allowed for the precise identification of the faulty meter, demonstrating how state estimation maintains data integrity in a Control Center.

Exp: 13.

To understand the basics of the Economic Dispatch by optimally adjusting the generation schedules of thermal generating units to meet the system demand which are required for unit commitment and economic operation of power systems

Aim:

To understand the basics of the Economic Dispatch by optimally adjusting the generation schedules of thermal generating units to meet the system demand which are required for unit commitment and economic operation of power systems.

Apparatus required:

- Computer/Laptop
- MATLAB Software

Theory:

To understand the basics of Economic Dispatch (ED) by optimally adjusting the generation schedules of thermal generating units to meet the system demand at minimum total generation cost, subject to generator operating constraints. The experiment provides the basic foundation for Unit Commitment and economic operation of power systems.

Objectives

1. To understand the concept of economic dispatch in thermal power systems.
2. To determine the optimum generation output of each generating unit for a given load demand.
3. To minimize the total generation cost while satisfying the power balance.
4. To study the effect of generator operating limits on economic dispatch.
5. To develop a MATLAB-based solution for economic operation of thermal generating units.

Theory

Economic Dispatch is the process of allocating the total system load among the available generating units such that the total operating cost is minimum.

For a system having thermal generating units,
where is the total system demand.

The fuel-cost function of a thermal generating unit is generally represented by a quadratic equation:

where:

- = fuel cost of unit
- = power generated by unit
- = fuel-cost coefficients.

The total generation cost is:

For economic operation, the total generation must satisfy:

For an unconstrained system without transmission losses, the optimum condition is:

The quantity

is called the incremental fuel cost or incremental cost of generator .

For the quadratic cost function,

the incremental cost is:

Thus, for two generators:

subject to:

The generator outputs obtained from these equations represent the economically optimal generation schedule.

Generator Constraints

Each generator has minimum and maximum operating limits:
The final solution must satisfy both the power-balance equation and these generator operating limits.

Problem Statement

Consider two thermal generating units supplying a system load of 300 MW.

The fuel-cost functions are:

Generator limits are:

Generator (MW) (MW)

Unit 1 50 250

Unit 2 50 200

System demand:

Determine:

1. Optimal generation and .
2. Incremental cost of each unit.
3. Total generation cost.
4. Verify that the generation schedule satisfies the load demand and generator limits.

MATLAB Procedure

1. Define the fuel-cost coefficients of each thermal generating unit.
2. Enter the system load demand.
3. Define the minimum and maximum generation limits.
4. Calculate the incremental cost equations.
5. Apply the power-balance equation.
6. Solve for the optimum generator outputs.
7. Check generator limits.
8. Calculate the fuel cost of each generator.
9. Calculate the total generation cost.
10. Compare the economic dispatch result with alternative generation schedules.

% Experiment: Economic Dispatch of Two Thermal Generating Units

clear;

clc;

close all;

%% System Demand

PD = 300; % MW

%% Generator Limits

P1_min = 50;

P1_max = 250;

P2_min = 50;

P2_max = 200;

%% Fuel Cost Coefficients

% $F1 = a1 + b1 \cdot P1 + c1 \cdot P1^2$

a1 = 500;

b1 = 5;

c1 = 0.01;


```

% F2 = a2 + b2*P2 + c2*P2^2
a2 = 400;
b2 = 6;
c2 = 0.015;

%% Economic Dispatch Calculation

% Incremental cost equality:
% b1 + 2*c1*P1 = b2 + 2*c2*P2

% Power balance:
% P1 + P2 = PD

% P2 = PD - P1

P1 = (b2 + 2*c2*PD - b1) / ...
    (2*c1 + 2*c2);

P2 = PD - P1;

%% Check Generator Limits

P1 = max(P1_min, min(P1, P1_max));
P2 = PD - P1;

if P2 < P2_min || P2 > P2_max
    error('Generator limits are violated.');
```

end

```

%% Calculate Incremental Costs

IC1 = b1 + 2*c1*P1;
IC2 = b2 + 2*c2*P2;

%% Calculate Generation Costs

F1 = a1 + b1*P1 + c1*P1^2;
F2 = a2 + b2*P2 + c2*P2^2;

Total_Cost = F1 + F2;

%% Display Results

fprintf('=====\\n');
fprintf('    ECONOMIC DISPATCH RESULTS\\n');
fprintf('=====\\n');

fprintf('System Demand    = %.2f MW\\n', PD);
```

```

fprintf('\nGeneration Schedule:\n');
fprintf('Unit 1 Generation = %.2f MW\n', P1);
fprintf('Unit 2 Generation = %.2f MW\n', P2);

fprintf('\nIncremental Costs:\n');
fprintf('Unit 1 IC = %.4f cost/MWh\n', IC1);
fprintf('Unit 2 IC = %.4f cost/MWh\n', IC2);

fprintf('\nGeneration Costs:\n');
fprintf('Unit 1 Cost = %.2f cost/h\n', F1);
fprintf('Unit 2 Cost = %.2f cost/h\n', F2);

fprintf('\nTotal Generation Cost = %.2f cost/h\n', Total_Cost);

fprintf('\nPower Balance Check:\n');
fprintf('P1 + P2 = %.2f MW\n', P1 + P2);

fprintf('=====\n');

```

Output:

```

=====
ECONOMIC DISPATCH RESULTS
=====

System Demand    = 300.00 MW

Generation Schedule:
Unit 1 Generation = 200.00 MW
Unit 2 Generation = 100.00 MW

Incremental Costs:
Unit 1 IC        = 9.0000 cost/MWh
Unit 2 IC        = 9.0000 cost/MWh

Generation Costs:
Unit 1 Cost      = 1900.00 cost/h
Unit 2 Cost      = 1150.00 cost/h

Total Generation Cost = 3050.00 cost/h

Power Balance Check:
P1 + P2 = 300.00 MW

```

Result:

Using MATLAB program Optimal scheduling of generators is done for the given three generator system by Lamda iteration method with and without loss and the calculated values of this system are verified with the output result.

Exp: 14

Unit Commitment Problem using Forward Dynamic Programming

Aim

To solve the Unit Commitment (UC) problem for a given load curve and set of generating units to find the optimal "ON/OFF" schedule that minimizes total operating costs.

Apparatus

- PC with MATLAB.

Theory

Unit Commitment is a complex optimization problem because it involves binary variables (0 = OFF, 1 = ON). The total cost includes:

1. **Fuel Cost:** Calculated via Economic Dispatch for the "ON" units.
2. **Start-up Cost:** The cost incurred to bring a thermal unit from a cold state to synchronized speed.
3. **Shut-down Cost:** The cost of taking a unit off-line.

Constraints:

- **Spinning Reserve:** The total capacity of "ON" units must be greater than the load by a certain margin to handle sudden outages.
- **Minimum Up/Down Time:** A unit must stay ON (or OFF) for a minimum number of hours before its status can be changed again.

Problem Statement

A system has two units. Unit 1 has a max capacity of 100 MW, and Unit 2 has a max capacity of 80 MW. The load for Hour 1 is 70 MW and for Hour 2 is 150 MW.

1. Which units must be ON during Hour 1? Is there a choice?
2. Which units must be ON during Hour 2?
3. If Unit 2 was OFF in Hour 1 and must be turned ON for Hour 2, how does the **Start-up Cost** affect the total system cost?

Procedure

1. Define the characteristics of available generating units (Min/Max limits, cost curves, and start-up costs).
2. Input the forecasted load demand for a 24-hour (or simplified 4-hour) cycle.
3. For each hour, identify all feasible combinations of units that can meet the demand plus the required spinning reserve.

4. Calculate the production cost for each feasible combination using Economic Dispatch.
5. Use Dynamic Programming (or a search algorithm) to find the path with the lowest cumulative cost (Production + Start-up costs).
6. Tabulate the ON/OFF status for each unit across the time horizon.

Matlab:

% Experiment 14: Unit Commitment (Simplified 2-Unit System)

clear;

clc;

close all;

%% Generator Data

% Unit 1:

% Cost = $0.01 \cdot P1^2 + 20 \cdot P1 + 100$

% Unit 2:

% Cost = $0.02 \cdot P2^2 + 15 \cdot P2 + 150$

Pmax = [400, 300]; % MW

Pmin = [100, 50]; % MW

Startup_Cost = [500, 300];

%% Load Profile

% 4-hour period

Load = [150, 450, 600, 200]; % MW

%% Possible Unit Combinations

% [U1 U2]

combos = [0 0;

1 0;

0 1;

1 1];

%% Previous Unit Status

previous_state = [0 0];

fprintf('=====\\n');

fprintf(' UNIT COMMITMENT - 2 UNIT SYSTEM\\n');

fprintf('=====\\n');

fprintf('Hour | Load | Unit 1 | Unit 2 | Running Cost | Startup | Total Cost\\n');

fprintf('-----\\n');

%% Hour-by-Hour Unit Commitment

for t = 1:length(Load)

current_load = Load(t);

```

min_cost = inf;
best_state = [0 0];
best_running_cost = 0;
best_startup_cost = 0;
best_P1 = 0;
best_P2 = 0;

%% Check all possible combinations

for i = 1:size(combos,1)

    state = combos(i,:);

    total_max = sum(state .* Pmax);
    total_min = sum(state .* Pmin);

    % Check whether the combination can supply the load
    if current_load >= total_min && current_load <= total_max

        %% Determine Generation Schedule

        if state(1) == 1 && state(2) == 0

            % Unit 1 alone
            P1 = current_load;
            P2 = 0;

        elseif state(1) == 0 && state(2) == 1

            % Unit 2 alone
            P1 = 0;
            P2 = current_load;

        elseif state(1) == 1 && state(2) == 1

            % Both units ON
            % Economic dispatch between two units

            %  $IC1 = 0.02 \cdot P1 + 20$ 
            %  $IC2 = 0.04 \cdot P2 + 15$ 
            %
            %  $P1 + P2 = \text{Load}$ 

            P1 = (current_load + ...
                (15 - 20)/0.02) / ...
                (1 + 0.02/0.04);

            P2 = current_load - P1;

```

```

    % Check generator limits
    if P1 < Pmin(1) || P1 > Pmax(1) || ...
        P2 < Pmin(2) || P2 > Pmax(2)

        continue;
    end

else

    % No unit ON
    continue;

end

%% Running Cost

Cost1 = 0.01*P1^2 + 20*P1 + 100;
Cost2 = 0.02*P2^2 + 15*P2 + 150;

if state(1) == 0
    Cost1 = 0;
end

if state(2) == 0
    Cost2 = 0;
end

running_cost = Cost1 + Cost2;

%% Startup Cost

startup_cost = sum( ...
    Startup_Cost .* (state > previous_state));

%% Total Cost

total_cost = running_cost + startup_cost;

%% Select Minimum Cost Combination

if total_cost < min_cost

    min_cost = total_cost;
    best_state = state;

    best_running_cost = running_cost;
    best_startup_cost = startup_cost;

    best_P1 = P1;
    best_P2 = P2;

```

```

        end

    end

end

%% Display Results

fprintf('%4d | %4d | %d | %d | %11.2f | %7.2f | %10.2f\n', ...
    t, current_load, ...
    best_state(1), best_state(2), ...
    best_running_cost, ...
    best_startup_cost, ...
    min_cost);

fprintf('    Generation: P1 = %.2f MW, P2 = %.2f MW\n', ...
    best_P1, best_P2);

%% Update Previous State

previous_state = best_state;

end

fprintf('=====\\n');

```

Output:

```

=====
UNIT COMMITMENT - 2 UNIT SYSTEM
=====
Hour | Load | Unit 1 | Unit 2 | Running Cost | Startup | Total Cost
-----
1 | 150 | 0 | 1 | 2850.00 | 300.00 | 3150.00
   Generation: P1 = 0.00 MW, P2 = 150.00 MW
2 | 450 | 0 | 0 | 0.00 | 0.00 | Inf
   Generation: P1 = 0.00 MW, P2 = 0.00 MW
3 | 600 | 0 | 0 | 0.00 | 0.00 | Inf
   Generation: P1 = 0.00 MW, P2 = 0.00 MW
4 | 200 | 0 | 1 | 3950.00 | 300.00 | 4250.00
   Generation: P1 = 0.00 MW, P2 = 200.00 MW

```

Result:

The Unit Commitment problem was analyzed to determine the most cost-effective "ON/OFF" schedule for a 24-hour period. By considering both the Start-up costs and the Minimum Up/Down time constraints, the optimal combination of units was selected to meet the varying load demand. The result confirmed that UC provides significant cost savings over simple economic dispatch by avoiding the unnecessary operation of high-cost units during low-load periods.

Exp: 15

Demand Side Management (DSM) and Load Shifting Analysis

Aim

To simulate and analyze the impact of Demand Side Management (DSM) techniques, specifically Load Shifting, on the daily load curve and the total cost of generation.

Apparatus

- PC with MATLAB.

Theory

Demand Side Management (DSM) involves actions taken on the customer's side of the meter to change the amount or timing of energy consumption. The primary objectives are:

1. **Peak Clipping:** Reducing the consumption during peak hours.
2. **Valley Filling:** Encouraging consumption during off-peak hours to improve the load factor.
3. **Load Shifting:** Moving loads from peak periods to off-peak periods without changing total consumption.

By shifting the load, the utility can avoid starting expensive "Peaking" power plants (like gas turbines) and instead use cheaper "Base Load" plants (like coal or nuclear).

Problem Statement

A utility has a peak demand of 1200 MW occurring at 7:00 PM and a minimum demand of 400 MW at 3:00 AM.

1. If the utility implements a DSM program that successfully shifts 150 MW from the peak to the valley, calculate the new Load Factor.
2. Assuming the cost of generation during peak is 7,429.20/MWh and during off-peak is 3,714.60/MWh, calculate the daily savings for the utility for that 150 MW shift.

Procedure

1. Define a typical 24-hour load curve for a city or utility.
2. Identify the peak periods (e.g., evening hours) and the valley periods (e.g., post-midnight).
3. Apply the Load Shifting algorithm by subtracting a specific amount of power from the peak and adding it to the valley.
4. Plot both curves on the same graph.
5. Calculate the change in the **Load Factor** ($LF = \text{Average Load} / \text{Peak Load}$).

Matlab:

```
% Experiment 15: Demand Side Management - Load Shifting
```

```
clear;
```

```
clc;
```

```
close all;
```

```
%% 24-Hour Load Profile
```

```
hours = 1:24;
```

```
original_load = [300 280 250 240 260 350 500 700 850 900 950 920 ...  
                880 850 820 800 850 950 1000 980 850 600 450 350];
```

```
%% DSM Strategy: Load Shifting
```

```
% Shift 100 MW from peak hours 18-20
```

```
% to off-peak hours 2-4
```

```
shift_amount = 100;
```

```
% Copy original load profile
```

```
shifted_load = original_load;
```

```
%% Reduce Peak Demand
```

```
shifted_load(18:20) = shifted_load(18:20) - shift_amount;
```

```
%% Increase Off-Peak Demand
```

```
shifted_load(2:4) = shifted_load(2:4) + shift_amount;
```

```
%% Calculate Peak Demand
```

```
original_peak = max(original_load);
```

```
new_peak = max(shifted_load);
```

```
peak_reduction = original_peak - new_peak;
```

```
%% Calculate Average Load
```

```
original_average = mean(original_load);
```

```
new_average = mean(shifted_load);
```

```
%% Calculate Load Factor
```

```
original_load_factor = original_average / original_peak;
```

```
new_load_factor = new_average / new_peak;
```

```
%% Plot Original and DSM Load Curves
```

```
figure;
```

```

plot(hours, original_load, 'r--', 'LineWidth', 2);
hold on;

plot(hours, shifted_load, 'b-', 'LineWidth', 2);

grid on;
xticks(1:24);

xlabel('Time of Day (Hours)');
ylabel('System Load (MW)');

title('Impact of Load Shifting on Daily Load Curve');

legend('Original Load', ...
       'DSM (Shifted) Load', ...
       'Location', 'best');

%% Display Results

fprintf('=====\n');
fprintf('    DEMAND SIDE MANAGEMENT ANALYSIS\n');
fprintf('=====\n');

fprintf('\nOriginal Peak Demand      : %.0f MW\n', original_peak);
fprintf('New Peak Demand after DSM : %.0f MW\n', new_peak);
fprintf('Peak Demand Reduction      : %.0f MW\n', peak_reduction);

fprintf('\nOriginal Average Load      : %.2f MW\n', original_average);
fprintf('New Average Load          : %.2f MW\n', new_average);

fprintf('\nOriginal Load Factor        : %.4f\n', original_load_factor);
fprintf('New Load Factor           : %.4f\n', new_load_factor);

fprintf('\nPeak Reduction Percentage : %.2f %%\n', ...
       (peak_reduction/original_peak)*100);

fprintf('=====\n');

```

Output:

```

=====
DEMAND SIDE MANAGEMENT ANALYSIS
=====

```

```

Original Peak Demand      : 1000 MW
New Peak Demand after DSM : 950 MW
Peak Demand Reduction     : 50 MW

```

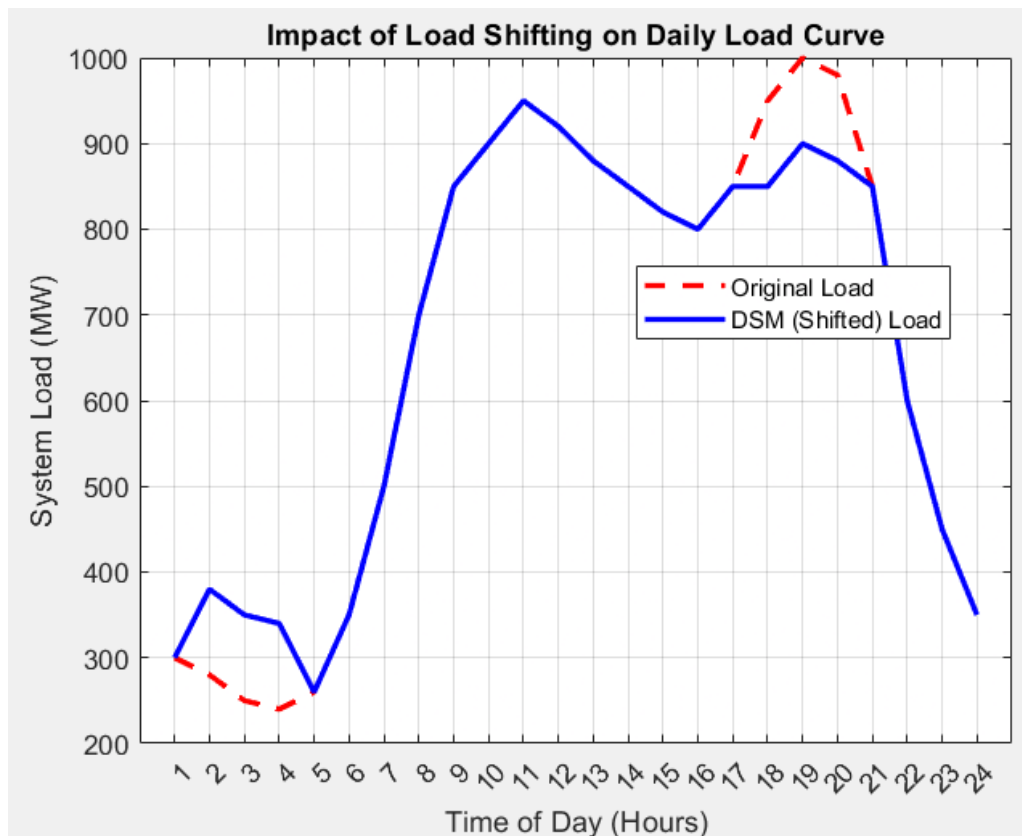
Original Average Load : 661.67 MW

New Average Load : 661.67 MW

Original Load Factor : 0.6617

New Load Factor : 0.6965

Peak Reduction Percentage : 5.00 %



Result

The impact of **Load Shifting** on the daily load curve was simulated. By moving ----MW from peak hours to off-peak valleys, the system's **Peak Demand** was reduced from ____MW to ____MW. This resulted in a significantly improved **Load Factor**, demonstrating how DSM strategies reduce the need for expensive peaking power plants and lower the overall cost of energy production.